St. Vincent's College Incorporated
Computer Engineering Department

LEVELING NEXUS

In Partial Fulfillment of the Requirements in Software Design for the Computer Engineering Course, St. Vincent's College Incorporated, Academic Year 2025-2026

Submitted by:
Nathaniel Joshua D. Yecpot
John Rick O. Lorejo

Submitted to:
Mrs. Christine June Jumawan

March 2026

TABLE OF CONTENTS

ACKNOWLEDGEMENT 2
INTRODUCTION 3
- Purpose of the System 3
- Scope and Limitations 3

REQUIREMENTS ANALYSIS 4
- Functional Requirements 4
- Non Functional Requirements 6
- User Roles 6

SYSTEM DESIGN 7
- Overall System Flow 7
- Feature Description 7

ARCHITURE DESIGN 8
DATABASE DESIGN 8
- Database Structure 8
- Data Organization Strategy 11
- Data Relationships 11
- Data Handling with Javascript 11
- Security and Data Integrity 11

SOFTWARE DOCUMENTATION 12
- Technical Documentation 12
- User Documentation 12
- Deployment Overview 12

IMPLEMENTATION 13
- Development Tools and Technologies 13
- System Development Approach 13
- Core Feature Implementation 13
- Core Structure Overview 14

TESTING PHASE 14
- Unit Testing 14
- Continuous Testing 14
- Acceptance Testing 14

FUTURE ENHANCEMENTS 15
CONCLUSION 15
REFERENCES 16

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to our Software Design Instructor, Mrs. Christine June Jumawan, for her invaluable guidance, expertise, and unwavering support throughout the semester. Her dedication to her profession and commitment to nurturing her students' skills have been truly exemplary and instrumental in our growth as developers.

We also extend our sincere appreciation to our project partner for their collaboration, dedication, and contributions, which were essential in completing Leveling Nexus.

Our deepest thanks go to our families and friends for their constant encouragement and support.

Lastly, we express our profound gratitude to God for His guidance, wisdom, and presence throughout the challenges and triumphs of this semester.

INTRODUCTION

Leveling Nexus is a gamified habit-tracking web application inspired by modern dungeon-style user interfaces seen in fantasy games. The system transforms daily tasks into quests, allowing users to gain experience points (XP) and improve virtual character statistics such as Strength, Intelligence, Stamina, and Health. Instead of simply checking off tasks, users experience personal growth through a structured and engaging progression system.

The application is designed to make productivity more interactive and rewarding. By combining task management with game mechanics, Leveling Nexus encourages users to stay consistent with their goals while enjoying a visually and conceptually immersive system.

Purpose of the System

The primary purpose of Leveling Nexus is to increase user motivation and consistency in completing daily habits and tasks. Many traditional habit-tracking applications lack engagement and emotional reward, which often leads users to abandon them. Leveling Nexus addresses this issue by introducing a progression-based reward system that gives immediate feedback and a sense of achievement.

The system aims to:

- Improve productivity
- Encourage discipline and consistency
- Make self-improvement enjoyable
- Provide measurable progress through XP, level and stat growth

Scope and Limitations

Leveling Nexus is designed as a web-based application accessible through modern web browsers. It allows users to create accounts, manage tasks, track progress, and monitor statistics. The system focuses on individual productivity and does not initially include advanced social features such as multiplayer interaction.

The system is limited to habit tracking, XP calculation, stat and level progression. It does not provide professional psychological assessment or medical advice. Future versions may be made accessible on mobile with its own UI/UX design.

REQUIREMENT ANALYSIS

Functional Requirements

The system shall allow users to register or login securely via email and password.

The stat points in the web application includes:

- Strength
- Intelligence
- Stamina
- Health

The system shall display:

- Current level
- Xp points
- Individual stats (Strength, Intelligence, Stamina, Health)

Quests in the web application are categorized into:

- Strength
- Intelligence

Stamina stats requires a gain of 3 Strength stats to gain 1 Stamina stats. Health stats requires a gain of 3 Strength stats, 3 Intelligence stats and 1 Stamina stat to gain 1 stat. Unless otherwise a quest built-in the system to gain immediately upon completion.

The quest system will include:

1. Daily Quest - These are simply checked based quests where the player gains xp accordingly after finishing a task.

Rules:

- The web app has a prepared daily quest already built in the system.
- The player can only create a maximum of 3 quests for himself.

Exp Mechanics:

- Each daily quest grants 50*(player level) xp points. The player also gains 1 stat point for the specific category the task is.
- The players loses 100*(player level) xp points per task uncompleted after the 24H mark finishes.

2. Active Quest - These are quests that are scheduled by time and day.

Rules:

- No quests are allowed to overlap or conflict with each other.
- Active Quest must be in intervals of 30mins to be allowed to be added e.g. 1H & 30min, 8:30pm-9:30pm, etc.
- Quest is categorised under Strength and Intelligence. Completing an active quest grants 1 stat point for the specific category.
- The quest is allowed to be accepted 30 minutes before the scheduled time. If a player forgets to accept it before the scheduled he loses xp points for accepting late, and point deduction will be implemented depending on how long the player is late.
- In order for the quest to finish, the finish button will be accessible 30 minutes before the deadline. The player must finish it on time otherwise the quest is considered failed and the player loses xp points.
- The xp are only added or deducted after the quest time is passed.

Exp Mechanics:

- For every 30 minutes, the quest gives 100*(player level) xp.
- If a player is late in accepting a quest, the player loses 200*(player level) xp points for the possible xp points gained in a quest. And for the next 30 minutes intervals of failing to accept the quest, the player loses 100*(player level) xp points per interval.
- The finish will be given a 30 minutes extra time after the deadline. If a player fails to finish on time, the player loses 200*(player level) xp points.
- If a player fails to accept a quest before 30 minutes of deadline, the player loses 400*(player level).

3. Urgent Quest - These are one-time checked based quests with a number of days set as a deadline.

Rules:

- The player can only add a maximum of 3 urgent quests per week.
- The player is allowed to add a maximum of 3 urgent quests a week ahead. These quests will only be officially added after the week ends and a new week starts.

Exp Mechanics:

- A player can only gain a maximum of 9000*(player level) xp points every week.
- Each urgent quest can gain a maximum of 3000*(player level) xp points upon completion.
- Each urgent quest calculates the xp gained by how many days it took for the task to be completed. After the first two days, the max exp gained for that quests will lose 500*(player level) xp points for every day passed without completion.

- If a player fails to finish an urgent quest in that week, the player loses 5000*(player level) xp.
- Upon completing an urgent quest, the player gains 1 point for all stat categories. Players can only gain a maximum of 3 stat points for each week.

The system shall store user data and task history for progress tracking. Users shall be able to view their past completed quests and overall improvement over time.

There shall be a global leaderboard which highlights the top 3 players and ranks the top 4-50 players.

There shall be a Rank Evaluation System to new players, which ask for the users habits, physical fitness, and mathematical skills where scores will decide the player level, and stat points for Strength and Intelligence where the new player starts.

Non Functional Requirements

The system must be user-friendly and responsive. It should load efficiently and maintain stable performance during normal usage. Security measures must protect user credentials and personal data.

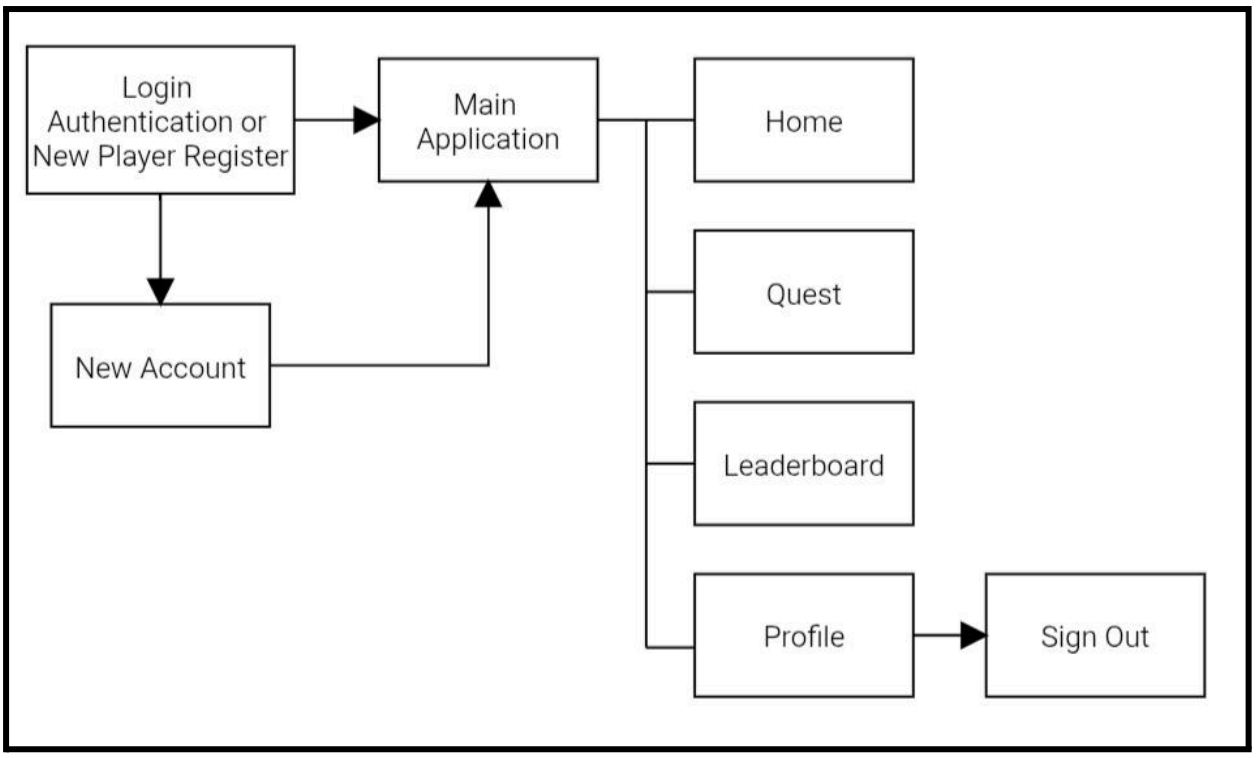
The application should be reliable and available with minimal downtime. It should maintain data integrity and prevent loss of user progress.

User Roles

The primary user role is the Registered User or The Player, who can manage tasks and track progress.

SYSTEM DESIGN

Overall System Flow



The system begins when a user logs into their account. After authentication, the home dashboard displays current statistics, XP, and active quests. There are 4 navigation buttons in the UI which includes Home, Quest, Leaderboard, Profile. Users may create new quests or complete existing ones in the Quest Page. Upon completion of a quest, XP is awarded, and the system recalculates the user's level and statistics.

The updated information is immediately reflected on the dashboard, providing instant feedback and motivation. If the User wishes to sign out his account, Profile Page shall have that logout button.

Feature Description

Key features of Leveling Nexus include:

- User authentication
- Quest management
- XP reward system
- Level progression system
- Stat growth system
- Activity Log

Each feature is designed to reinforce positive habits and provide visual representation of improvement.

ARCHITECTURE DESIGN

Leveling Nexus follows a 3-tier architecture model consisting of:

- **Presentation Layer** – The user interface displayed in the web browser.
- **Application Layer** – The backend logic that processes XP calculations, level progression, and stat updates.
- **Data Layer** – The database that stores user accounts, quests, XP, and statistics.

This architecture separates responsibilities, making the system scalable and easier to maintain. The presentation layer handles user interaction, the application layer manages logic and rules, and the data layer ensures secure storage of information.

The system may use standard web technologies, specifically HTML, CSS, JavaScript for the frontend, and a Backend Assistance Service, specifically Firebase for database and JavaScript backend processing.

DATABASE DESIGN

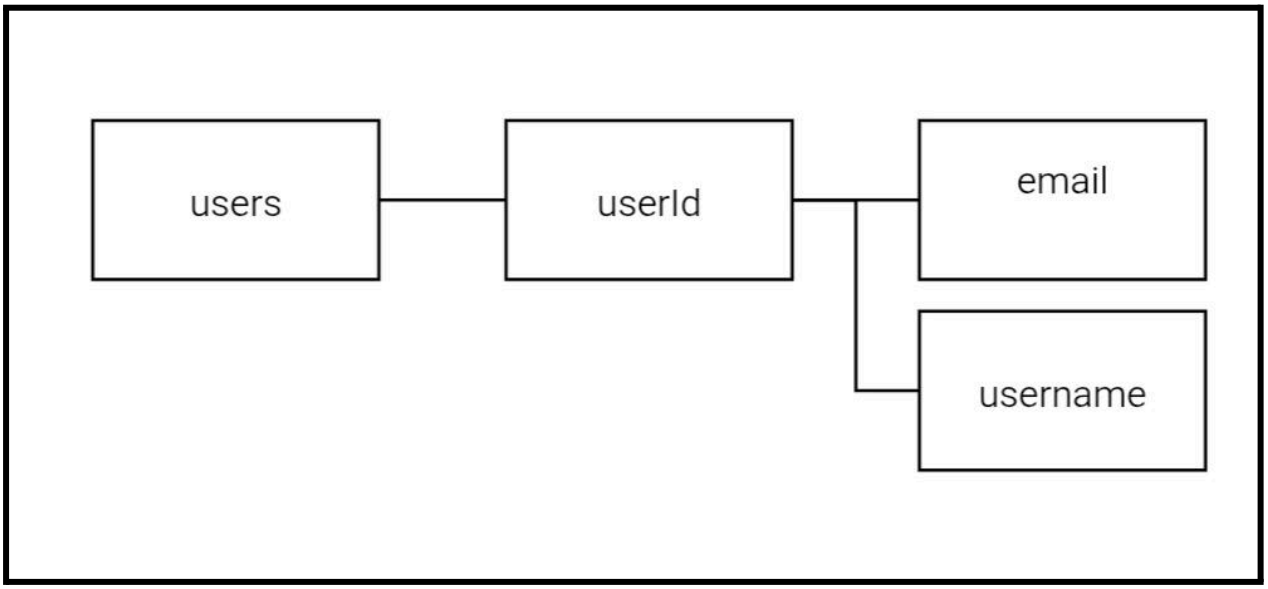
Leveling Nexus uses Firebase Cloud Firestore as its primary database system. Firestore is a cloud-based NoSQL database that stores data in collections and documents instead of traditional relational tables. This structure allows flexible, scalable, and real-time data management suitable for web applications.

The system interacts with the database using JavaScript through the Firebase SDK. All user data, quests, experience points (XP), and character statistics are stored and retrieved dynamically using asynchronous JavaScript functions.

Database Structure

The database is organized into collections. Each collection contains documents representing individual records.

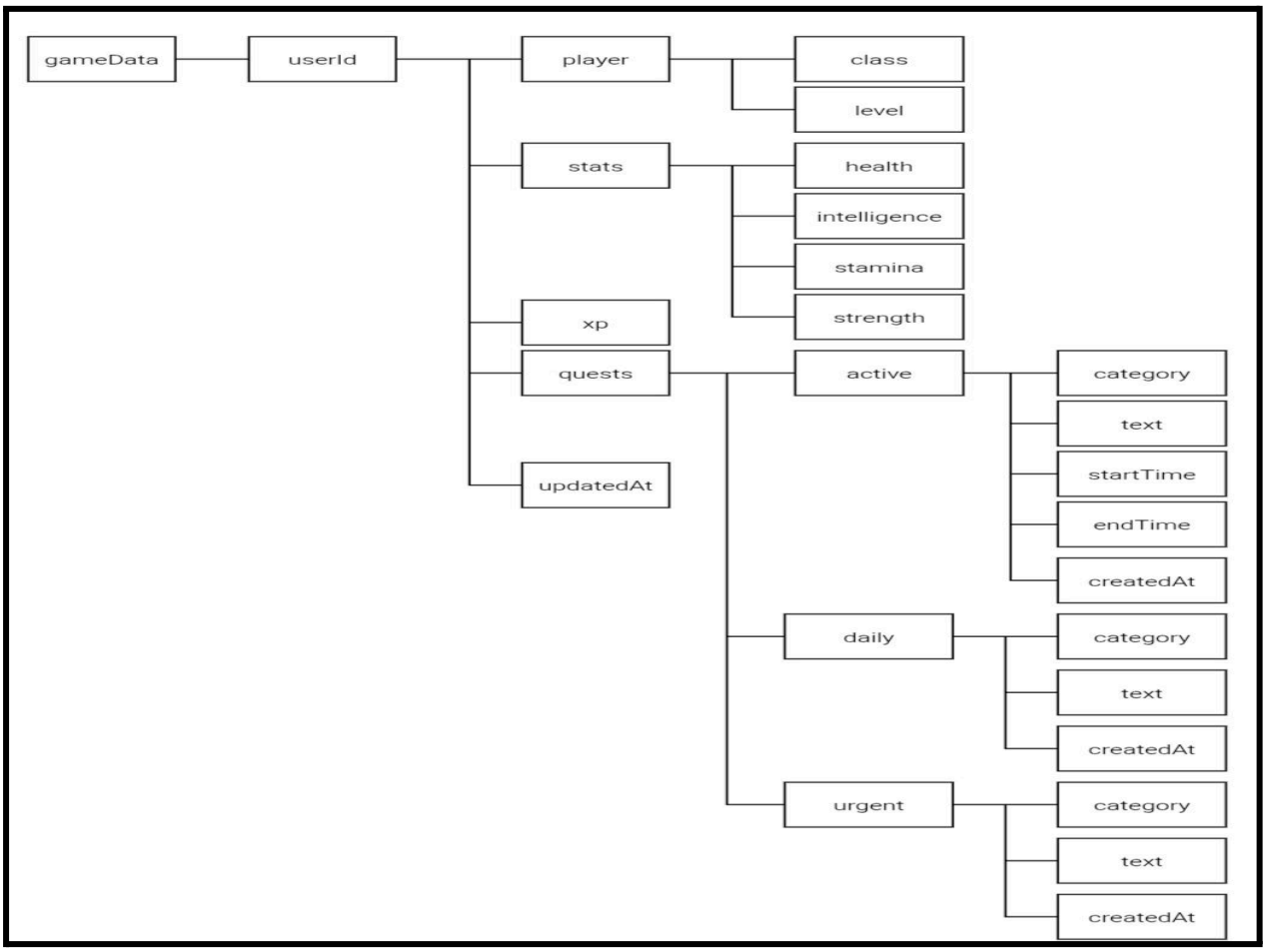
1. **Users Collection**



Each document represents a registered user and contains:

- email
- username

2. **GameData Collection**



Each document includes the following fields:

1. player (map)
    - class (string)
    - level (number)

2. stats (map)
    - health (number)
    - intelligence (number)
    - stamina (number)
    - strength (number)

3. xp (number) - Stores the total experience points earned by the player.

4. quests (map) - This field contains categorized quest arrays:
    - active (array)

Each quest inside the array include:

- category (string)
- text (string)
- startTime (number)
- endTime (number)
- createdAt (number) - Stores the last update timestamp for synchronization and tracking.

    - daily (array)

Each quest inside the array include:

- category (string)
- text (string) - Quest description
- createdAt (number) - Stores the last update timestamp for synchronization and tracking.

    - urgent (array)

Each quest inside the array include:

- category (string)
- text (string)
- createdAt (number)

5. updatedAt (number) - Stores the last update timestamp for synchronization and tracking.

The database structure is designed to be flexible and scalable. As the development of Leveling Nexus progresses, additional documents and fields may be introduced to support new features and system enhancements. This includes potential expansions such as social features, advanced progression systems, and

extended user data. The use of Firebase Cloud Firestore allows seamless modification of the database without requiring major structural changes.

Data Organization Strategy

Instead of separating quests into a different collection, Leveling Nexus embeds quest data directly inside the user's gameData document. This design simplifies data retrieval because all player-related information can be accessed in a single database read.

Since Firestore does not use foreign keys, relationships are handled implicitly within the document structure. Each gameData document contains all information necessary to represent a complete player state.

Data Relationships

Since Firestore is a NoSQL database, relationships are handled using document references rather than foreign keys. Each quest stores the userId of its owner, allowing the system to retrieve only quests belonging to a specific user.

This structure ensures efficient querying and scalability without requiring complex table joins.

Data Handling with JavaScript

JavaScript is used to perform database operations such as:

- Creating user accounts
- Adding new quests
- Updating quest completion status
- Calculating and updating XP
- Updating user statistics
- All operations are performed using Firebase's asynchronous functions to ensure smooth and real-time user experience.

Security and Data Integrity

Firebase Authentication is utilized to secure user accounts and ensure that only authenticated users can access their respective data. Firestore security rules are implemented to control and restrict unauthorized read and write operations. During account registration, email verification is required, and strong password policies are enforced to enhance security and protect user information.

This approach ensures data privacy, integrity, and secure storage of user progress.

SOFTWARE DOCUMENTATION

This section describes how the Leveling Nexus system operates and how users interact with its features. It includes both technical details and general usage guidance.

Technical Documentation

Leveling Nexus is a web-based application developed using JavaScript and Firebase services. Firebase Authentication is used to manage user login and account security, while Cloud Firestore stores all user-related data such as experience points (XP), stats, and quests.

All player data is organized within the gameData collection, where each user has a document containing their level, XP, stats, and categorized quests. JavaScript handles system processes such as updating XP, calculating levels, and modifying player statistics based on completed tasks.

The system is designed for efficiency by grouping all player-related data into a single document, allowing fast data retrieval and real-time updates.

User Documentation

Leveling Nexus does not include a built-in tutorial. Instead, the system is designed with a simple and intuitive interface that allows users to understand its functionality through direct interaction.

Users begin by registering an account and verifying their email. After logging in, they are presented with a dashboard displaying their level, XP, and stats. Users can add quests, organize them into categories such as active, daily, or urgent, and mark them as completed.

When a quest is completed, the system automatically updates XP and player stats. Changes are reflected immediately, allowing users to track their progress in real time.

The absence of a tutorial encourages users to explore the system naturally, relying on clear layout and straightforward features for ease of use.

Deployment Overview

Leveling Nexus is deployed as a web-based application and can be accessed through modern web browsers without requiring installation.

Firebase services handle backend functionality, including authentication and database operations. This setup simplifies deployment and allows the system to scale efficiently as more users are added.

The application is designed to be responsive and accessible across various devices, including desktops and mobile devices.

IMPLEMENTATION

The implementation phase focuses on the actual development of the Leveling Nexus web application using modern web technologies and cloud services.

Development Tools and Technologies

The system is developed using JavaScript as the main programming language for handling application logic and user interaction. Firebase services are used for backend support, including Firebase Authentication for user login and Cloud Firestore for database management.

The user interface is built using standard web technologies such as HTML and CSS, ensuring a responsive and visually engaging design inspired by game-like interfaces.

System Development Approach

The system is developed incrementally, where features are implemented step by step. Core functionalities such as user authentication, quest creation, XP calculation, and stat updates are built first before adding enhancements.

After each feature is implemented, testing is performed to ensure correct functionality and to prevent errors from affecting other parts of the system.

Core Feature Implementation

Key features of Leveling Nexus are implemented using JavaScript and Firebase integration:

User Authentication – Users can register and log in using Firebase Authentication with email verification.

Quest Management – Users can create, categorize, and manage quests stored in Firestore under the gameData collection.

XP and Level System – Completing quests grants XP, which updates the user’s level dynamically.

Stat System – Player stats such as Strength, Intelligence, Stamina, and Health increase based on completed tasks.

Real-Time Updates – Changes in quests, XP, and stats are reflected immediately using Firebase's real-time capabilities.

Code Structure Overview

The system follows a modular structure where different functionalities are separated into logical sections. JavaScript functions handle database operations, user input, and UI updates.

Firebase configuration and database access are managed separately to maintain clean and organized code. This structure improves readability, maintainability, and scalability of the system.

TESTING PHASE

The testing phase ensures that the Leveling Nexus system functions correctly, efficiently, and provides a positive user experience. Multiple testing methods are applied throughout development to identify and resolve issues.

Unit Testing

Unit testing is conducted after implementing individual features or components. Each function, such as XP calculation, quest creation, and stat updates, is tested independently to ensure correct behavior. This helps detect errors early in the development process.

Continuous Testing

The system is tested after every completed coding session. This approach ensures that newly added features do not break existing functionality. Continuous testing allows for immediate debugging and maintains system stability throughout development.

Acceptance Testing

Acceptance testing is performed by allowing other users, such as friends or peers, to interact with the system. This testing focuses on user experience, usability, and overall satisfaction. Feedback gathered from users is used to improve the interface, functionality, and engagement of the application.

FUTURE ENHANCEMENTS

Leveling Nexus has significant potential for expansion as development continues. Future updates aim to enhance user engagement, social interaction, and overall system functionality.

Planned enhancements include:

- **Chat Messaging System** – Allows users to communicate and share progress with others in real time.
- **In-Game Community Forum** – A space where users can discuss strategies, share achievements, and motivate each other.
- **Guilds or Groups System** – Enables users to form teams, complete group quests, and participate in collaborative challenges.
- **Achievement and Badge System** – Rewards users for reaching milestones and completing special tasks..

These enhancements aim to transform Leveling Nexus from a personal productivity tool into a more interactive and community-driven platform.

CONCLUSION

Leveling Nexus presents a creative approach to habit tracking by integrating gamification elements into a productivity system. By transforming daily tasks into quests and rewarding users with experience points and stat progression, the system encourages consistency, motivation, and personal growth.

The application demonstrates how modern web technologies, such as JavaScript and Firebase, can be used to build an interactive and responsive system. Through its structured design, including requirement analysis, system architecture, and database implementation, Leveling Nexus provides a functional and scalable solution for managing daily habits.

Despite its simplicity, the system effectively achieves its main goal of making productivity more engaging. The intuitive interface allows users to navigate and use the application without the need for a built-in tutorial, relying instead on clear design and immediate feedback.

Overall, Leveling Nexus highlights the potential of combining game-inspired mechanics with real-world productivity tools. With further development and additional features, the system can evolve into a more advanced and community-driven platform.

REFERENCES

Firebase Documentation. (https://firebase.google.com/docs)

MDN Web Docs – JavaScript. (https://developer.mozilla.org/)

W3Schools – Web Development Tutorials. (https://www.w3schools.com/)